

PROJECT REPORT

Edge Detector Comparison & Analysis Tool

Submitted by:

Aditi Soni – 24BAI10490

B.Tech CSE (AI/ML)

Abstract

Edge detection is an important part of Computer Vision because it helps in finding boundaries and important changes in an image. Different edge detection methods work differently depending on the image and the amount of noise present in it.

In this project, four classical edge detection methods are compared: **Sobel, Canny, Laplacian of Gaussian (LoG), and Difference of Gaussians (DoG)**. The project is implemented in Python and works completely through the command line. Along with detecting edges from a normal image, the project also studies how Gaussian noise affects the output of these methods.

For the experiment, Gaussian noise is added at different levels and each edge detector is applied to the resulting images. Runtime, edge density and percentage change in edge density are recorded for every method. The project also generates comparison images, a CSV file containing experimental results and a noise robustness plot.

The experiment shows that different edge detectors behave differently when noise is increased. In the selected experiment, Sobel showed a very large increase in detected edge pixels after noise was added, while Canny showed relatively small changes at lower noise levels. LoG and DoG also showed increasing changes as the noise level became higher. These results demonstrate the importance of selecting an appropriate edge detection method according to the image and noise conditions.

1. Introduction

Computer Vision deals with the processing and understanding of digital images. One of the basic tasks in Computer Vision is identifying the boundaries of objects present in an image. These boundaries are called **edges**.

An edge is generally found where there is a significant change in pixel intensity. For example, the boundary between a black object and a white background produces a strong intensity change. Detecting such boundaries can help in further tasks such as image segmentation, object detection, shape recognition and feature extraction.

There are many edge detection techniques available. Some methods are simple gradient-based techniques, while others use multiple processing stages to obtain cleaner edges.

In this project, four classical techniques are studied and compared:

- Sobel
- Canny
- Laplacian of Gaussian (LoG)
- Difference of Gaussians (DoG)

The project also focuses on the effect of image noise. Noise can create unwanted intensity changes, which may be incorrectly detected as edges. Therefore, the behavior of each method is studied by adding different levels of Gaussian noise to the input image.

2. Problem Statement

The objective of this project is to develop a command-line Computer Vision tool that can apply multiple classical edge detection techniques to the same image and compare their outputs.

The project should also examine how the performance of these methods changes when Gaussian noise is introduced into the image.

The comparison is performed using:

- Visual edge maps
- Runtime
- Edge density
- Percentage change in edge density
- Noise robustness analysis

3. Objectives

The main objectives of the project are:

1. To understand the basic concept of edge detection in Computer Vision.
2. To implement the Sobel edge detector using NumPy-based convolution.
3. To apply Canny, LoG and DoG edge detection methods.
4. To compare the output of all four methods on the same image.
5. To study the effect of Gaussian noise on edge detection.
6. To measure runtime for different methods.

7. To calculate edge density for clean and noisy images.
8. To calculate the change in edge density as noise increases.
9. To generate visual and numerical results automatically.
10. To keep the complete project executable from the command line without requiring a GUI.

4. Background and Theory

4.1 Edge Detection

Edge detection is the process of identifying areas in an image where the intensity changes significantly.

Consider an image containing an object against a plain background. The boundary of the object normally produces a sharp change in intensity. An edge detector attempts to identify this change.

Edge detection is commonly used as an early processing step because it reduces an image to important structural information instead of processing every pixel in the same way.

4.2 Sobel Edge Detector

The Sobel operator is a gradient-based edge detection method.

It uses two convolution kernels:

- One kernel detects changes in the horizontal direction.
- The other kernel detects changes in the vertical direction.

The two gradient values are combined to estimate the strength of an edge.

In this project, the basic Sobel convolution is implemented manually using NumPy. This was useful for understanding how convolution-based edge detection works instead of directly treating the detector as a black-box operation.

Sobel is simple and computationally straightforward. However, because noise itself can create sudden changes in pixel intensity, noisy images can produce many unwanted responses.

4.3 Canny Edge Detector

Canny is a multi-stage edge detection method.

The main stages of Canny include:

1. Gaussian smoothing to reduce noise.
2. Calculation of image gradients.
3. Non-maximum suppression to make edges thinner.
4. Double thresholding.
5. Edge tracking using hysteresis.

Because of these stages, Canny generally produces thin and well-defined edges.

The smoothing stage also helps reduce the effect of small variations caused by noise. However, very high noise levels can still affect the final result.

4.4 Laplacian of Gaussian (LoG)

Laplacian of Gaussian combines two operations:

1. Gaussian smoothing.
2. Laplacian-based edge detection.

The Gaussian filter smooths the image before the Laplacian operator detects rapid intensity changes.

This is useful because directly applying a second-order derivative to a noisy image can produce many unwanted responses. Smoothing before the Laplacian operation helps reduce some of these effects.

The amount of smoothing can influence the final edge detection result.

4.5 Difference of Gaussians (DoG)

Difference of Gaussians works by creating two blurred versions of an image using different Gaussian scales and subtracting one from the other.

The subtraction emphasizes structures that change between the two scales.

DoG is related to scale-space processing and can highlight boundaries and local image structures.

The result depends on the selected Gaussian scales, so different parameter values can produce different edge maps.

5. Methodology

The project follows a simple processing pipeline.

Step 1: Input Image

The program receives an image path through the command line.

Example:

```
python main.py --image sample_images/synthetic_test.png --output results
```

Step 2: Edge Detection

The input image is processed using four methods:

- Sobel
- Canny
- LoG
- DoG

Step 3: Individual Results

The output of each method is saved separately.

Step 4: Comparison Grid

The four outputs are combined into a side-by-side comparison image.

Step 5: Noise Experiment

Gaussian noise is added to the original image at different noise levels.

The experiment uses:

- $\sigma = 0$
- $\sigma = 5$
- $\sigma = 15$
- $\sigma = 25$
- $\sigma = 40$

The clean image ($\sigma = 0$) is used as the baseline.

Step 6: Measurement

For every method and noise level, the following values are recorded:

- Runtime

- Edge density
- Percentage change in edge density

Step 7: Result Generation

The results are stored in CSV format and a noise robustness plot is generated.

6. Technologies Used

The project was developed using the following technologies:

Python

Python was used as the main programming language because it provides useful libraries for image processing and numerical operations.

OpenCV

OpenCV was used for image processing and standard computer vision operations such as Canny edge detection and filtering.

NumPy

NumPy was used for numerical operations and for implementing the Sobel convolution.

Matplotlib

Matplotlib was used to generate plots and visualize the experimental results.

CSV

CSV format was used to store runtime, edge density and density-change measurements.

7. Project Structure

The final project contains the following files and folders:

edge-detector-comparison/

|

├── .gitignore

├── edge_detectors.py

├── main.py

└── requirements.txt

```
├── README.md
├── REPORT.md
|
├── sample_images/
|   └── synthetic_test.png
|
└── results/
    ├── edges_sobel.png
    ├── edges_canny.png
    ├── edges_log.png
    ├── edges_dog.png
    ├── comparison_grid.png
    ├── noise_experiment.csv
    ├── noise_robustness_plot.png
    └── noisy test images
```

8. Implementation

The project is divided mainly into two parts.

8.1 edge_detectors.py

This file contains the implementation of the edge detection methods.

The Sobel operation is implemented using NumPy-based convolution, while Canny, LoG and DoG processing use the required image-processing operations.

8.2 main.py

main.py acts as the main entry point of the project.

It handles:

- Command-line arguments
- Image loading

- Running edge detectors
- Saving output images
- Noise generation
- Runtime measurement
- Edge density calculation
- CSV generation
- Plot generation

- **9. Execution**

-
- The project can be executed from a terminal.
- First, the required Python packages are installed using:
- `pip install -r requirements.txt`
- The main project command is:
- `python main.py --image sample_images/synthetic_test.png --output results`
- The program successfully completed execution and generated the required output files inside the `results` directory.
- **10. Experimental Setup**
-
- To study noise robustness, Gaussian noise was added to the input image.
- The following noise levels were tested:

Noise Level (σ)	Description
0	Clean image
5	Low noise
15	Moderate noise
25	Higher noise
40	High noise

- For each noise level, all four edge detectors were applied.
- The clean image was used as the baseline for calculating percentage change in edge density.

11. Results and Analysis

The clean image produced the following edge densities:

Method	Edge Density
Sobel	0.0369
Canny	0.0172
LoG	0.0708
DoG	0.0967

The values are different because each edge detector uses a different approach for identifying edges.

11.1 Results for Different Noise Levels

Noise σ	Method	Runtime (ms)	Edge Density	Density Change
0	Sobel	3777.86	0.0369	0.00%
0	Canny	3482.25	0.0172	0.00%
0	LoG	3639.60	0.0708	0.00%
0	DoG	3883.88	0.0967	0.00%
5	Sobel	3883.98	0.9786	2553.75%
5	Canny	3499.60	0.0175	1.91%
5	LoG	3544.80	0.0727	2.74%
5	DoG	3613.24	0.0977	1.04%
15	Sobel	3651.85	0.9959	2600.91%
15	Canny	3753.20	0.0175	1.94%
15	LoG	3514.55	0.0757	6.98%
15	DoG	3544.42	0.0992	2.68%
25	Sobel	4353.75	0.9977	2605.74%
25	Canny	5917.11	0.0176	2.51%
25	LoG	6682.68	0.0903	27.52%
25	DoG	6515.98	0.1069	10.57%
40	Sobel	7100.85	0.9982	2606.94%
40	Canny	7060.20	0.0874	407.93%
40	LoG	4402.31	0.1376	94.34%
40	DoG	4047.14	0.1341	38.74%

12. Noise Robustness Analysis

The experiment shows a clear difference in the behavior of the four methods.

Sobel

Sobel showed a very large increase in edge density even at a low noise level.

For the clean image, its edge density was approximately **0.0369**. At $\sigma = 5$, it increased to approximately **0.9786**.

The density change was approximately **2553.75%**.

At higher noise levels, the edge density remained close to 1.0. This means that a very large portion of the pixels were classified as edges in this particular experiment.

Canny

Canny showed relatively small changes at lower noise levels.

At $\sigma = 5$, its density change was approximately **1.91%**.

At $\sigma = 15$, the change was approximately **1.94%**, and at $\sigma = 25$ it was approximately **2.51%**.

However, at $\sigma = 40$, the density change increased substantially to approximately **407.93%**.

This shows that Canny handled the lower noise levels more consistently in this experiment, but very high noise still affected the result.

LoG

LoG showed a gradual increase in edge density as noise increased.

The density change was:

- 2.74% at $\sigma = 5$
- 6.98% at $\sigma = 15$
- 27.52% at $\sigma = 25$
- 94.34% at $\sigma = 40$

This indicates that the effect of noise became much more noticeable at higher noise levels.

DoG

DoG also showed relatively small changes at lower noise levels.

Its density change was:

- 1.04% at $\sigma = 5$
- 2.68% at $\sigma = 15$
- 10.57% at $\sigma = 25$
- 38.74% at $\sigma = 40$

The change became larger as the noise level increased.

13. Runtime Analysis

Runtime was also recorded for each method.

The runtime values varied depending on the method and noise level. For example, on the clean image:

- Sobel: approximately 3777.86 ms
- Canny: approximately 3482.25 ms
- LoG: approximately 3639.60 ms
- DoG: approximately 3883.88 ms

At higher noise levels, the runtime also changed.

These measurements should not be considered fixed benchmarks because runtime depends on the computer hardware, Python environment, image size and implementation details.

The main purpose of recording runtime in this project was to provide an additional comparison between the methods under the same experimental conditions.

14. Visual Results

The project generates visual results for all four methods.

The main comparison image is:

Figure 1: Comparison of Sobel, Canny, LoG and DoG edge detection methods.

The project also generates a noise robustness plot:

Figure 2: Noise robustness analysis at different Gaussian noise levels.

These generated images are stored in the results directory of the project repository.

The comparison grid makes it easier to observe the difference between the edge maps, while the noise robustness plot shows how the measured edge density changes as noise increases.

15. Discussion

The experiment demonstrates that no edge detector behaves exactly the same under all image conditions.

Sobel is relatively simple and is useful for understanding gradient-based edge detection. However, in this experiment its response was strongly affected by the added Gaussian noise.

Canny produced a comparatively stable edge density at the lower noise levels. This can be related to its multi-stage processing, including Gaussian smoothing and thresholding. At the highest noise level tested, however, its output was also affected significantly.

LoG and DoG showed gradual changes at lower noise levels and larger changes as the noise became stronger.

One important point is that **edge density should not be treated as a direct measure of accuracy**. A higher edge density does not automatically mean that more correct edges were detected. For measuring actual detection accuracy, a labelled ground-truth edge image would be required.

Therefore, the results in this project are mainly useful for understanding the behavior of the methods under the selected experimental conditions.

16. Limitations

The project has some limitations:

1. The experiment was performed using a limited test image rather than a large dataset.
2. Edge density does not directly measure whether the detected edges are correct.
3. The results depend on the parameters used by each edge detector.
4. Runtime can change depending on the computer and software environment.
5. Only Gaussian noise was considered in the noise experiment.
6. The project focuses on classical edge detection methods and does not compare them with modern deep-learning-based edge detectors.

17. Applications

Edge detection is used in many areas of Computer Vision.

Some possible applications include:

- Object boundary detection
- Image segmentation
- Shape recognition
- Industrial inspection
- Medical image processing
- Document analysis
- Road and lane detection
- Feature extraction
- Object detection systems

For example, in industrial inspection, edges can help identify cracks or boundaries in manufactured parts. In document processing, edges can help identify the boundaries of text and objects.

18. Future Scope

The project can be extended in several ways.

18.1 Larger Dataset

The experiment can be performed on a larger collection of real-world images instead of only one test image.

18.2 Ground-Truth Evaluation

A labelled edge dataset can be used to calculate metrics such as:

- Precision
- Recall
- F1-score

This would provide a more direct measurement of edge detection accuracy.

18.3 Parameter Tuning

The project can be extended to allow users to change parameters such as Gaussian kernel size, thresholds and smoothing values directly from the command line.

18.4 More Edge Detectors

Additional methods such as Prewitt, Roberts, Scharr and other techniques can be included for a broader comparison.

18.5 Different Types of Noise

Future experiments can include salt-and-pepper noise, speckle noise and other real-world noise conditions.

18.6 Learning-Based Methods

The classical methods can also be compared with modern deep-learning-based edge detection methods.

19. Conclusion

This project implemented and compared four classical edge detection methods: Sobel, Canny, Laplacian of Gaussian and Difference of Gaussians.

The project was developed as a command-line Computer Vision application and successfully generated individual edge maps, a comparison grid, a CSV file containing experimental measurements and a noise robustness plot.

The noise experiment showed that the four methods respond differently when Gaussian noise is introduced. Sobel showed a particularly large increase in detected edge density in the selected experiment. Canny remained comparatively stable at lower noise levels, while LoG and DoG showed gradually increasing changes as the noise level increased. At very high noise, all four methods were affected to some extent.

Overall, the project helped in understanding both the working principles of classical edge detectors and the practical effect of image noise on their outputs. It also showed why an edge detection method should be evaluated according to the characteristics of the image and the conditions in which the system will be used.

20. References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson.
2. J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
3. OpenCV Documentation – Image Processing and Edge Detection.
4. NumPy Documentation – Numerical Computing with Python.
5. Matplotlib Documentation – Data Visualization and Plotting.